Desarrollo de aplicaciones móviles I

Tema Nº9:
Fragments y Recycler View

Indicador de logro Nº9:
Utiliza Fragments desarrollando interfaces gráficas que ocupen un área mayor a la pantalla del dispositivo móvil

idat

TEMA Nº9:
Fragments y Recycler View.

Subtema 9.1:
Creando nuestros fragments
Ejemplos:

Ahora procederemos a crear nuestro Fragment donde mostraremos nuestro RecyclerView con la lista de pizzas. Para empezar, vamos a agregar directamente el RecyclerView a nuestro layout.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <android.support.v7.widget.RecyclerView
        android:id="@+id/recyclerview"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        tools:itemCount="5"
        tools:layoutManager="android.support.v7.widget.LinearLayoutManager"
        tools:listitem="@layout/item_row"
        tools:spanCount="2" />

</LinearLayout>
```

Java

```java
public class MainFragment extends Fragment {

    public static final String BASE_URL = "https://private-3cc5a4-codingpizza.apiary-mock.com";
    private static final String TAG = "MainFragment";

    public MainFragment() {
    }

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_main, container, false);
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        mRecyclerView = view.findViewById(R.id.recyclerview);
    }

}
```

Subtema 9.2:
¿Cómo funciona un RecyclerView?
Ejemplos:

Luego procederemos a crear el layout para la fila que mostrará la información de cada pizza.

```xml
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <ImageView
        android:id="@+id/ivPizzaImage"
        android:layout_width="100dp"
        android:layout_height="100dp"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

```

```
17      <TextView
18          android:id="@+id/tvName"
19          android:layout_width="wrap_content"
20          android:layout_height="wrap_content"
21          android:layout_marginStart="16dp"
22          android:layout_marginTop="16dp"
23          android:text="Napolitana"
24          android:textSize="18sp"
25          android:textStyle="bold"
26          app:layout_constraintStart_toEndOf="@+id/ivPizzaImage"
27          app:layout_constraintTop_toTopOf="parent" />
28
29      <TextView
30          android:id="@+id/tvDescription"
31          android:layout_width="0dp"
32          android:layout_height="wrap_content"
33          android:layout_marginBottom="16dp"
34          android:ellipsize="end"
35          android:maxLines="4"
36          android:text="TextView"
37          app:layout_constraintBottom_toBottomOf="parent"
38          app:layout_constraintEnd_toEndOf="parent"
39          app:layout_constraintHorizontal_bias="1.0"
40          app:layout_constraintStart_toStartOf="@+id/tvName"
41          app:layout_constraintTop_toBottomOf="@+id/tvName" />
42  </android.support.constraint.ConstraintLayout>
```

Creando nuestro POJO

Como ya se ha mencionado en guías anteriores, para crear un objeto a partir de un JSON es necesario realizar un Plain Old Java Object. Para esto crearemos nuestra clase Pizza, que es muy parecida a la que hicimos anteriormente solo que en esta oportunidad tiene una url donde está almacenada la imagen.

Java

```java
public class Pizza {

    @SerializedName("name")
    @Expose
    private String name;
    @SerializedName("id")
    @Expose
    private Integer id;
    @SerializedName("description")
    @Expose
    private String description;
    @SerializedName("image")
    @Expose
    private String image;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getImage() {
        return image;
    }

    public void setImage(String image) {
        this.image = image;
    }
}
```

Creando nuestro CustomAdapter y ViewHolder

Ahora crearemos nuestro CustomAdapter el cual extiende de RecyclerView.Adapter y vamos a crear dentro del mismo una clase con un viewHolder. Dentro de este último vamos a asignar la información de cada pizza.

```java
public class CustomAdapter extends RecyclerView.Adapter<CustomAdapter.MyViewHolder> {
    private List<Pizza> mNameList;

    public static class MyViewHolder extends RecyclerView.ViewHolder {
        public ImageView mPizzaImage;
        public TextView mPizzaName, mPizzaDescription;

        public MyViewHolder(View itemView) {
            super(itemView);
            mPizzaName = itemView.findViewById(R.id.tvName);
            mPizzaDescription = itemView.findViewById(R.id.tvDescription);
            mPizzaImage = itemView.findViewById(R.id.ivPizzaImage);
        }
    }

    public CustomAdapter(List<Pizza> pizzaList) {
        this.mNameList = pizzaList;
    }
```

```java
    @Override
    public MyViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View itemView = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.item_row, parent, false);
        return new MyViewHolder(itemView);
    }

    @Override
    public void onBindViewHolder(MyViewHolder holder, int position) {
        holder.mPizzaName.setText(mNameList.get(position).getName());
        holder.mPizzaDescription.setText(mNameList.get(position).getDescription());
    }

    @Override
    public int getItemCount() {
        return mNameList.size();
    }
}
```

Subtema 9.3:

Implementando RecyclerView

Ejemplos:

Como en todas las aplicaciones que creamos, necesitamos empezar agregando las dependencias a nuestro build.gradle.

```
dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation 'com.android.support:appcompat-v7:28.0.0'
    implementation 'com.android.support.constraint:constraint-layout:1.1.3'
    testImplementation 'junit:junit:4.12'
    androidTestImplementation 'com.android.support.test:runner:1.0.2'
    androidTestImplementation 'com.android.support.test.espresso:espresso-core:3.0.2'
    implementation 'com.android.support:recyclerview-v7:28.0.0'
    implementation 'com.squareup.retrofit2:retrofit:2.5.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.5.0'
    implementation 'com.github.bumptech.glide:glide:4.8.0'
}
```

Subtema 9.3:

Glide

Ejemplos:

Para poder mostrar las imágenes de cada pizza necesitaremos utilizar Glide. Usar Glide es bastante sencillo solo debemos indicarle en cuál View va a buscar la ImageView en la que queremos mostrar la imagen. with(), luego desde dónde va a cargar la imagen con el método .load(), y finalmente, con el método .into() pasamos nuestro ImageView quedando de la siguiente manera:

Java

```
@Override
public void onBindViewHolder(MyViewHolder holder, int position) {
    holder.mPizzaName.setText(mNameList.get(position).getName());
    holder.mPizzaDescription.setText(mNameList.get(position).getDescription());
    Glide.with(holder.itemView)
    .load(mNameList.get(position).getImage())
            .into(holder.mPizzaImage);
}
```

Obteniendo datos con Retrofit

Antes de crear el RecyclerView vamos a empezar creando la llamada a la API para obtener la información que le vamos a pasar a este. ;ya que esta url contiene la listas de pizzas con imágenes.

Java

```java
public interface PizzaService {
    @GET("/v2/pizzas")
    Call<List<Pizza>> listPizzas();
}
```

Rellenando el RecyclerView

Para llenar nuestra lista de pizzas vamos a necesitar crear en nuestro Fragment, un RecyclerView y una instancia de Retrofit. Luego de esto vamos a crear un Service con Retrofit que nos permitirá realizar una llamada a la URL que ya hemos definido previamente y al obtener los datos vamos a asignarlos a nuestro nuevo CustomAdapter.

Java

```java
public class MainFragment extends Fragment {

    public static final String BASE_URL = "https://private-3cc5a4-codingpizza.apiary-mock.com";
    private RecyclerView mRecyclerView;
    private static final String TAG = "MainFragment";
    private CustomAdapter mAdapter;

    public MainFragment() {
    }

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_main, container, false);
    }
```

```java
    private void getPizzaList(Retrofit retrofit) {
        PizzaService pizzaService = retrofit.create(PizzaService.class);
        Call<List<Pizza>> pizzaList = pizzaService.listPizzas();
        pizzaList.enqueue(new Callback<List<Pizza>>() {
            @Override
            public void onResponse(Call<List<Pizza>> call, Response<List<Pizza>> response) {
                if (response.isSuccessful()) {
                    mAdapter = new CustomAdapter(response.body());
                    RecyclerView.LayoutManager mLayoutManager = new LinearLayoutManager(getActivity());
                    mRecyclerView.addItemDecoration(new DividerItemDecoration(getActivity(), LinearLayoutManager.VERTICAL));
                    mRecyclerView.setAdapter(mAdapter);
                    mRecyclerView.setLayoutManager(mLayoutManager);
                } else {
                    Log.e(TAG, "onResponse: Llamada fallida");
                }
            }

            @Override
            public void onFailure(Call<List<Pizza>> call, Throwable t) {
                Log.e(TAG, "onFailure: Ha fallado la llamada");
            }
        });
    }
```

```java

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        mRecyclerView = view.findViewById(R.id.recyclerview);
        Retrofit retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(GsonConverterFactory.create())
                .build();

        getPizzaList(retrofit);
    }

```

```java

}
```

Con esto ya tendremos nuestra simple aplicación para ver Pizzas y sus ingredientes.

Actividad:

a) CUESTIONARIO TÉCNICO

- ¿Cuál es el impacto de utilizar o no POJOS al trabajar con archivos JSON en el contexto de la presente guía?
- Detalle su opinión respecto a utilizar Glide, ¿es una buena práctica?, desarrolle la pregunta
- Es una tendencia moderna de desarrollo tener una sola activity con varios fragments; ¿dónde está la ventaja de este paradigma de desarrollo?
- Indique en qué archivo se agregan las dependencias externas

- ¿En qué situaciones aplica el uso de Fragments?

b) CONCLUSIONES DE LA EXPERIENCIA

_____
_____
_____
_____
_____
_____
_____